

```
In [1]: import os
os.environ["OMP_NUM_THREADS"] = "1"
import flexynesis
import torch
torch.set_num_threads(4)
```

Seed set to 42

```
In [2]: # parameters cell (required to pass arguments to the notebook) (see View
HPO_ITER = 5 # number of HPO iterations for final modeling run
```

Modeling Breast Cancer Subtypes

Here, we demonstrate the capabilities of `flexynesis` on a multi-omic dataset of Breast Cancer samples from the [METABRIC consortium](#). The data was downloaded from [Cbioportal](#) and randomly split into `train` (70% of the samples) and `test` (30% of the samples) data folders. The data files were processed to follow the same nomenclature.

- `gex.csv` contains "gene expression" data
- `cna.csv` contains "copy number alteration" data
- `mut.csv` contains "mutation" data, which is a binary matrix of genes versus samples.
- `clin.csv` contains "clinical/sample metadata", which is a table of clinical parameters such as age, gender, therapy, subtypes.

Data Download

The data can be downloaded as follows:

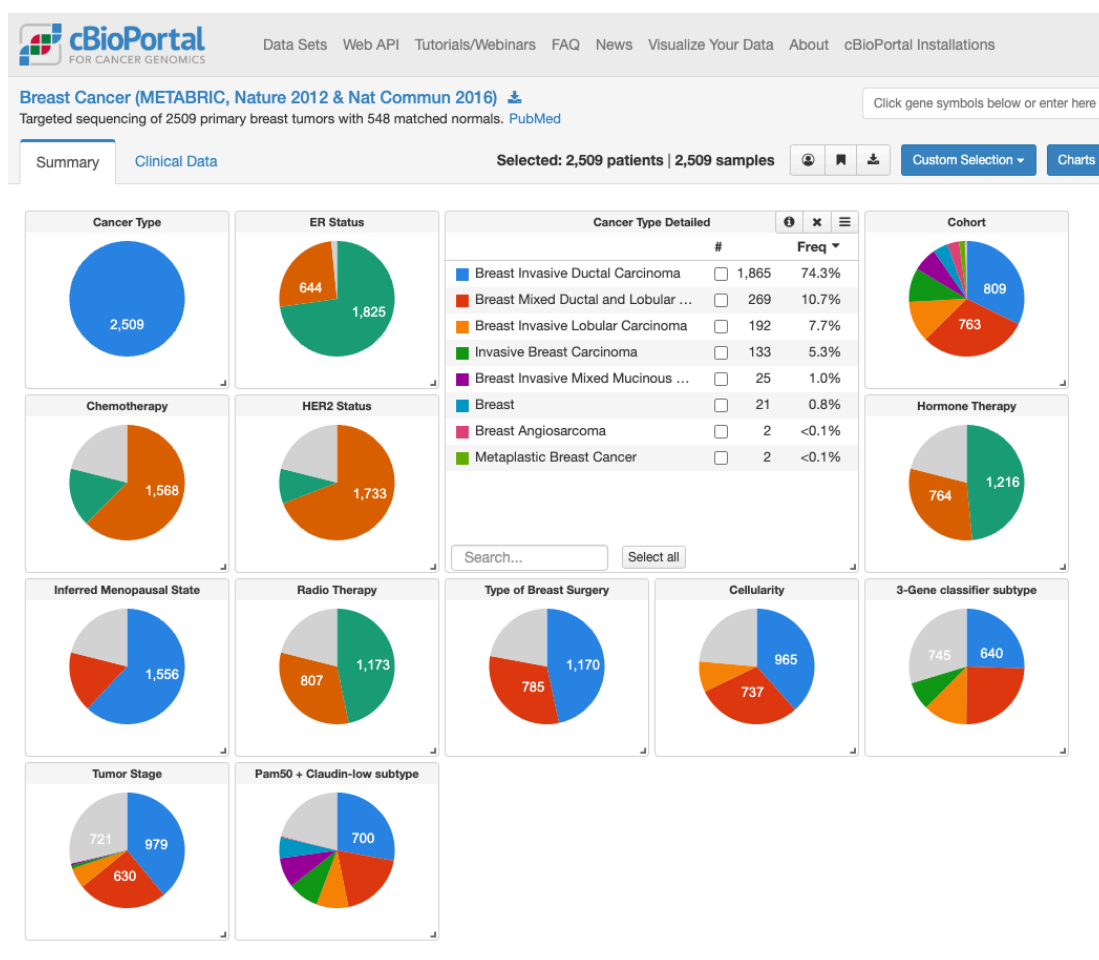
```
In [3]: if not os.path.exists("brca_metabric_processed"):
!wget -O brca_metabric.tgz "https://bimsbstatic.mdc-berlin.de/akalin/
```

```
--2025-03-12 19:52:03-- https://bimsbstatic.mdc-berlin.de/akalin/buyar/fl
exynesis-benchmark-datasets/brca_metabric_processed.tgz
141.80.181.46, 141.80.181.47rlin.de (bimsbstatic.mdc-berlin.de)...
connected. to bimsbstatic.mdc-berlin.de (bimsbstatic.mdc-berlin.de)|141.8
0.181.46|:443...
HTTP request sent, awaiting response... 200 OK
Length: 407225158 (388M) [application/octet-stream]
Saving to: 'brca_metabric.tgz'
```

```
brca_metabric.tgz 100%[=====>] 388.36M 11.2MB/s in 35s
```

```
2025-03-12 19:52:39 (11.1 MB/s) - 'brca_metabric.tgz' saved [407225158/407
225158]
```

```
brca_metabric_processed/
brca_metabric_processed/test/
brca_metabric_processed/test/gex.csv
brca_metabric_processed/test/mut.csv
brca_metabric_processed/test/clin.csv
brca_metabric_processed/test/cna.csv
brca_metabric_processed/9606.protein.aliaes.v12.0.txt.gz
brca_metabric_processed/9606.protein.links.v12.0.txt.gz
brca_metabric_processed/train/
brca_metabric_processed/train/gex.csv
brca_metabric_processed/train/mut.csv
brca_metabric_processed/train/clin.csv
brca_metabric_processed/train/cna.csv
```



Let's check the number of samples and number of features in the corresponding files under train and test folders:

```
In [4]: # train data
!wc -l ./brca_metabric_processed/train/*

1307 ./brca_metabric_processed/train/clin.csv
22543 ./brca_metabric_processed/train/cna.csv
20604 ./brca_metabric_processed/train/gex.csv
174 ./brca_metabric_processed/train/mut.csv
44628 total
```

```
In [5]: # test data
!wc -l ./brca_metabric_processed/test/*

561 ./brca_metabric_processed/test/clin.csv
22543 ./brca_metabric_processed/test/cna.csv
20604 ./brca_metabric_processed/test/gex.csv
174 ./brca_metabric_processed/test/mut.csv
43882 total
```

Importing Multiomics Data Into Flexynesis

Procedure

We use the `flexynesis.DataImporter` class to import multiomics data from the data folders. Data importing includes:

1. Validation of the data folders
2. Reading data matrices
3. Data processing, which includes:
 - Cleaning up the data matrices to:
 - remove uninformative features (e.g. features with near-zero-variation)
 - remove samples with too many NA values
 - remove features with too many NA values and impute NA values for the rest with few NA values
4. Feature selection **only on training data** for each omics layer separately:
 - Features are sorted by Laplacian score
 - Features that make it in the `top_percentile`
 - Highly redundant features are further removed (for a pair of highly correlated features, keep the one with the higher laplacian score).
5. Harmonize the training data with the test data.
 - Subset the test data features to those that are kept for training data
6. Normalize the datasets
 - Normalize training data (standard scaling) and apply the same scaling factors to the test data.
7. (Optional): Log transform the final matrices.
8. Distinguish numerical and categorical variables in the "clin.csv" file. For categorical variables, create a numerical encoding of the labels for training data. Use the same encoders to map the test samples to the same numerical encodings.

Usage

- Here, we import both train/test datasets from the data folder we downloaded and unpacked before.
- We choose which omic layers to import
- We choose whether we want to concatenate the data matrices (early integration) or not (intermediate integration) before running them through the neural networks.
- We want to apply feature selection and keep only top 10% of the features. In the end, we want to keep at least 1000 features per omics layer.
- We apply a variance threshold (for simplicity of demonstration, we want to keep a small number of most variable features). Setting this to 80, will remove 80% of the features with lowest variation from each modality.

```
In [6]: data_importer = flexynesis.DataImporter(path='./brca_metabric_processed/  
                                              data_types = ['gex', 'cna'],  
                                              concatenate=False,  
                                              top_percentile=10,  
                                              min_features=100,  
                                              variance_threshold=0.8, # set to  
                                              )  
train_dataset, test_dataset = data_importer.import_data()
```

```

[INFO] ===== Importing Data =====
[INFO] Validating data folders...

[INFO] ----- Reading Data -----
[INFO] Importing ./brca_metabric_processed/train/gex.csv...
[INFO] Importing ./brca_metabric_processed/train/cna.csv...
[INFO] Importing ./brca_metabric_processed/train/clin.csv...

[INFO] ----- Reading Data -----
[INFO] Importing ./brca_metabric_processed/test/gex.csv...
[INFO] Importing ./brca_metabric_processed/test/cna.csv...
[INFO] Importing ./brca_metabric_processed/test/clin.csv...

[INFO] ----- Checking for problems with the input data -----
[INFO] Data structure is valid with no errors or warnings.

[INFO] ----- Processing Data (train) -----

[INFO] ----- Cleaning Up Data -----

[INFO] working on layer: gex
[INFO] Imputing NA values to median of features, affected # of cells in the matrix 7 # of rows: 5
[INFO] Number of NA values: 0
[INFO] DataFrame gex - Removed 16482 features.

[INFO] working on layer: cna
[INFO] Imputing NA values to median of features, affected # of cells in the matrix 108 # of rows: 87
[INFO] Number of NA values: 0
[INFO] DataFrame cna - Removed 18033 features.
[INFO] DataFrame gex - Removed 3 samples (0.23%).
[INFO] DataFrame cna - Removed 3 samples (0.23%).
[INFO] Implementing feature selection using laplacian score for layer: gex with 4121 features and 1303 samples
Calculating Laplacian scores: 100%|██████████| 4121/4121 [00:07<00:00, 571.6 6it/s]
Filtering redundant features: 100%|██████████| 412/412 [00:00<00:00, 15539.6 3it/s]
[INFO] Implementing feature selection using laplacian score for layer: cna with 4509 features and 1303 samples
Calculating Laplacian scores: 100%|██████████| 4509/4509 [00:07<00:00, 575.8 0it/s]
Filtering redundant features: 100%|██████████| 450/450 [00:00<00:00, 745433.1 8it/s]

```

```
[INFO] ----- Processing Data (test) -----

[INFO] ----- Cleaning Up Data -----

[INFO] working on layer: gex
[INFO] Number of NA values: 0
[INFO] DataFrame gex - Removed 16482 features.

[INFO] working on layer: cna
[INFO] Imputing NA values to median of features, affected # of cells in the matrix 63 # of rows: 51
[INFO] Number of NA values: 0
[INFO] DataFrame cna - Removed 18033 features.
[INFO] DataFrame gex - Removed 2 samples (0.36%).
[INFO] DataFrame cna - Removed 2 samples (0.36%).

[INFO] ----- Harmonizing Data Sets -----

[INFO] ----- Finished Harmonizing -----

[INFO] ----- Normalizing Data -----

[INFO] ----- Normalizing Data -----
[INFO] Training Data Stats: {'feature_count in: gex': 408, 'feature_count in: cna': 450, 'sample_count': 1303}
[INFO] Test Data Stats: {'feature_count in: gex': 408, 'feature_count in: cna': 450, 'sample_count': 558}
[INFO] Merging Feature Logs...
[INFO] Data import successful.
```

- **dataset.dat** contains the data matrices

```
In [7]: train_dataset.dat
```

```
Out[7]: {'gex': tensor([[ 0.2835, -0.2079,  0.8022, ..., -0.5616, -0.3481, -0.5837],
                        [ 0.5312,  0.9580,  1.3312, ..., -0.6958, -1.1154, -0.9176],
                        [ 0.3158,  0.2672,  1.2577, ..., -0.6923, -1.1226, -0.4238],
                        ...,
                        [-2.5582, -1.8169, -1.6219, ...,  2.3811,  1.9294,  2.3498],
                        [ 0.3855,  0.0501,  0.2753, ..., -0.6017, -0.5617,  0.3110],
                        [ 0.3399,  0.6494,  0.7000, ...,  1.0998,  1.0932,  1.2755]]),
         'cna': tensor([[0.2908, 0.2509, 0.2583, ..., 0.2730, 0.2473, 0.2537],
                        [1.5840, 1.5381, 1.5378, ..., 1.5572, 1.5310, 1.5447],
                        [0.2908, 0.2509, 0.2583, ..., 0.2730, 0.2473, 0.2537],
                        ...,
                        [1.5840, 1.5381, 1.5378, ..., 0.2730, 0.2473, 1.5447],
                        [0.2908, 0.2509, 0.2583, ..., 0.2730, 0.2473, 0.2537],
                        [0.2908, 0.2509, 0.2583, ..., 0.2730, 0.2473, 0.2537]])})
```

```
In [8]: train_dataset.dat['gex'].shape, train_dataset.dat['cna'].shape
```

```
Out[8]: (torch.Size([1303, 408]), torch.Size([1303, 450]))
```

- **dataset.ann** contains the sample annotation data (from clin.csv), where the keys are variable names and values are tensors.

```
In [9]: train_dataset.ann
```

```
Out[9]: {'LYMPH_NODES_EXAMINED_POSITIVE': tensor([1, 0, 2, ..., 0, 0, 0]),
        'NPI': tensor([5.0800, 2.0440, 4.0360, ..., 4.0640, 4.0300, 2.0260],
                      dtype=torch.float64),
        'AGE_AT_DIAGNOSIS': tensor([77.1300, 81.8800, 70.5200, ..., 68.1600, 7
2.1800, 76.9300],
                      dtype=torch.float64),
        'OS_MONTHS': tensor([126.8667, 122.7000, 123.5333, ..., 16.7000, 139.
6333, 117.5667],
                      dtype=torch.float64),
        'RFS_MONTHS': tensor([125.2000, 121.0900, 121.9100, ..., 8.3200, 13
7.8000, 115.2600],
                      dtype=torch.float64),
        'CELLULARITY': tensor([0., 0., 0., ..., 2., 0., 2.], dtype=torch.float
64),
        'CHEMOTHERAPY': tensor([0., 0., 0., ..., 0., 0., 0.], dtype=torch.floa
t64),
        'COHORT': tensor([2., 0., 3., ..., 2., 0., 2.], dtype=torch.float64),
        'ER_IHC': tensor([1., 1., 1., ..., 0., 0., 1.], dtype=torch.float64),
        'HER2_SNP6': tensor([0., 2., 2., ..., 2., 2., 2.], dtype=torch.float6
4),
        'HORMONE_THERAPY': tensor([1., 1., 1., ..., 0., 0., 1.], dtype=torch.f
loat64),
        'INFERRED_MENOPAUSAL_STATE': tensor([0., 0., 0., ..., 0., 0., 0.], dty
pe=torch.float64),
        'SEX': tensor([0., 0., 0., ..., 0., 0., 0.], dtype=torch.float64),
        'INTCLUST': tensor([ 7.,  9., 10., ...,  1.,  2.,  3.], dtype=torch.fl
oat64),
        'OS_STATUS': tensor([0., 1., 0., ..., 1., 1., 1.], dtype=torch.float6
4),
        'CLAUDIN_SUBTYPE': tensor([3., 2., 3., ..., 0., 3., 2.], dtype=torch.f
loat64),
        'THREEGENE': tensor([0., 1., 0., ..., nan, 0., 1.], dtype=torch.float6
4),
        'VITAL_STATUS': tensor([2., 1., 2., ..., 1., 1., 0.], dtype=torch.floa
t64),
        'LATERALITY': tensor([1., 0., nan, ..., 0., nan, 0.], dtype=torch.floa
t64),
        'RADIO_THERAPY': tensor([0., 1., 1., ..., 0., 0., 0.], dtype=torch.flo
at64),
        'HISTOLOGICAL_SUBTYPE': tensor([0., 0., 4., ..., 0., 0., 0.], dtype=to
rch.float64),
        'BREAST_SURGERY': tensor([1., 1., 0., ..., 1., 0., 0.], dtype=torch.fl
oat64),
        'RFS_STATUS': tensor([0., 0., 0., ..., 1., 0., 1.], dtype=torch.float6
4)}
```

- A mapping of the sample labels for categorical variables can be found in `dataset.label_mappings`

```
In [10]: train_dataset.label_mappings
```

```

Out[10]: {'CELLULARITY': {0: 'High', 1: 'Low', 2: 'Moderate', 3: nan},
'CHEMOTHERAPY': {0: 'NO', 1: 'YES'},
'COHORT': {0: 'cohort1',
1: 'cohort2',
2: 'cohort3',
3: 'cohort4',
4: 'cohort5'},
'ER_IHC': {0: 'Negative', 1: 'Positive', 2: nan},
'HER2_SNP6': {0: 'GAIN', 1: 'LOSS', 2: 'NEUTRAL', 3: 'UNDEF'},
'HORMONE_THERAPY': {0: 'NO', 1: 'YES'},
'INFERRED_MENOPAUSAL_STATE': {0: 'Post', 1: 'Pre'},
'SEX': {0: 'Female'},
'INTCLUST': {0: '1',
1: '10',
2: '2',
3: '3',
4: '4ER+',
5: '4ER-',
6: '5',
7: '6',
8: '7',
9: '8',
10: '9'},
'OS_STATUS': {0: '0:LIVING', 1: '1:DECEASED'},
'CLAUDIN_SUBTYPE': {0: 'Basal',
1: 'Her2',
2: 'LumA',
3: 'LumB',
4: 'NC',
5: 'Normal',
6: 'claudin-low'},
'THREEGENE': {0: 'ER+/HER2- High Prolif',
1: 'ER+/HER2- Low Prolif',
2: 'ER-/HER2-',
3: 'HER2+',
4: nan},
'VITAL_STATUS': {0: 'Died of Disease',
1: 'Died of Other Causes',
2: 'Living',
3: nan},
'LATERALITY': {0: 'Left', 1: 'Right', 2: nan},
'RADIO_THERAPY': {0: 'NO', 1: 'YES'},
'HISTOLOGICAL_SUBTYPE': {0: 'Ductal/NST',
1: 'Lobular',
2: 'Medullary',
3: 'Metaplastic',
4: 'Mixed',
5: 'Mucinous',
6: 'Other',
7: 'Tubular/ cribriform',
8: nan},
'BREAST_SURGERY': {0: 'BREAST CONSERVING', 1: 'MASTECTOMY', 2: nan},
'RFS_STATUS': {0: '0:Not Recurred', 1: '1:Recurred', 2: nan}}

```

- As the data matrices are stored as tensors, the row and column names cannot be stored as tensors. These are stored in the same dataset object as: `dataset.samples` and `dataset.features`


```
In [11]: train_dataset.samples[1:10], train_dataset.features
```

```
Out[11]: ([ 'MB-0348',
            'MB-7086',
            'MB-0593',
            'MB-4994',
            'MB-7152',
            'MB-6218',
            'MB-0482',
            'MB-5482',
            'MB-5653'],
          {'gex': Index(['FOXA1', 'MLPH', 'ESR1', 'GATA3', 'SPDEF', 'TBC1D9', 'FO
XC1', 'C1S',
                        'XBP1', 'CA12',
                        ...,
                        'N4BP2', 'TNFSF14', 'LEP', 'INIP', 'RPL7L1', 'MBD4', 'HCG2P7',
'ZNF430',
                        'KIAA1791', 'IL10'],
                        dtype='object', length=408),
          'cna': Index(['MST01', 'FCRLA', 'TOP1P1', 'LAMC1', 'TDRKH', 'MST02P',
'DAP3',
                        'YY1AP1', 'SLC50A1', 'EFNA1',
                        ...,
                        'XPR1', 'SOAT1', 'SELENBP1', 'PI4KB', 'SELP', 'RFX5', 'AXDND1',
'KIAA1614', 'TRMT1L', 'FM09P'],
                        dtype='object', length=450)})
```

- We can get a summary of sample metadata using `print_summary_stats`. For categorical variables, we can the sample counts per label and for numerical variables, we get mean/median statistics.

```
In [12]: flexynesis.print_summary_stats(train_dataset)
```

Summary for variable: LYMPH_NODES_EXAMINED_POSITIVE

Numerical Variable Summary: Median = 0.0, Mean = 1.9286262471220261

Summary for variable: NPI

Numerical Variable Summary: Median = 4.04, Mean = 4.017291158864159

Summary for variable: AGE_AT_DIAGNOSIS

Numerical Variable Summary: Median = 61.79, Mean = 61.30643898695318

Summary for variable: OS_MONTHS

Numerical Variable Summary: Median = 114.4666667, Mean = 125.03573804066693

Summary for variable: RFS_MONTHS

Numerical Variable Summary: Median = 100.63, Mean = 109.94034535686878

Summary for variable: CELLULARITY

Categorical Variable Summary:

Label: High, Count: 656

Label: Low, Count: 136

Label: Moderate, Count: 484

Label: nan, Count: 27

Summary for variable: CHEMOTHERAPY

Categorical Variable Summary:

Label: NO, Count: 1044

Label: YES, Count: 259

Summary for variable: COHORT

Categorical Variable Summary:

Label: cohort1, Count: 308

Label: cohort2, Count: 196

Label: cohort3, Count: 521

Label: cohort4, Count: 159

Label: cohort5, Count: 119

Summary for variable: ER_IHC

Categorical Variable Summary:

Label: Negative, Count: 289

Label: Positive, Count: 994

Label: nan, Count: 20

Summary for variable: HER2_SNP6

Categorical Variable Summary:

Label: GAIN, Count: 279

Label: LOSS, Count: 67

Label: NEUTRAL, Count: 955

Label: UNDEF, Count: 2

Summary for variable: HORMONE_THERAPY

Categorical Variable Summary:

Label: NO, Count: 508

Label: YES, Count: 795

Summary for variable: INFERRED_MENOPAUSAL_STATE

Categorical Variable Summary:

Label: Post, Count: 1034

Label: Pre, Count: 269

Summary for variable: SEX

Categorical Variable Summary:

Label: Female, Count: 1303

Summary for variable: INTCLUST

Categorical Variable Summary:

Label: 1, Count: 93
Label: 10, Count: 151
Label: 2, Count: 50
Label: 3, Count: 195
Label: 4ER+, Count: 154
Label: 4ER-, Count: 50
Label: 5, Count: 127
Label: 6, Count: 60
Label: 7, Count: 129
Label: 8, Count: 197
Label: 9, Count: 97

Summary for variable: OS_STATUS

Categorical Variable Summary:

Label: 0:LIVING, Count: 539
Label: 1:DECEASED, Count: 764

Summary for variable: CLAUDIN_SUBTYPE

Categorical Variable Summary:

Label: Basal, Count: 145
Label: Her2, Count: 152
Label: LumA, Count: 468
Label: LumB, Count: 328
Label: NC, Count: 5
Label: Normal, Count: 90
Label: claudin-low, Count: 115

Summary for variable: THREEGENE

Categorical Variable Summary:

Label: ER+/HER2- High Prolif, Count: 414
Label: ER+/HER2- Low Prolif, Count: 425
Label: ER-/HER2-, Count: 197
Label: HER2+, Count: 128
Label: nan, Count: 139

Summary for variable: VITAL_STATUS

Categorical Variable Summary:

Label: Died of Disease, Count: 432
Label: Died of Other Causes, Count: 331
Label: Living, Count: 539
Label: nan, Count: 1

Summary for variable: LATERALITY

Categorical Variable Summary:

Label: Left, Count: 649
Label: Right, Count: 588
Label: nan, Count: 66

Summary for variable: RADIO_THERAPY

Categorical Variable Summary:

Label: NO, Count: 522
Label: YES, Count: 781

Summary for variable: HISTOLOGICAL_SUBTYPE

Categorical Variable Summary:

```

Label: Ductal/NST, Count: 999
Label: Lobular, Count: 98
Label: Medullary, Count: 18
Label: Metaplastic, Count: 1
Label: Mixed, Count: 135
Label: Mucinous, Count: 15
Label: Other, Count: 12
Label: Tubular/ cribriform, Count: 15
Label: nan, Count: 10

```

```

-----
Summary for variable: BREAST_SURGERY
Categorical Variable Summary:
  Label: BREAST CONSERVING, Count: 524
  Label: MASTECTOMY, Count: 763
  Label: nan, Count: 16

```

```

-----
Summary for variable: RFS_STATUS
Categorical Variable Summary:
  Label: 0:Not Recurred, Count: 763
  Label: 1:Recurred, Count: 539
  Label: nan, Count: 1

```

Training flexynesis models

We create a `tuner` object by specifying:

1. `dataset` : the training dataset (as we constructed above)
 2. `model_class` : which model architecture to use: a) DirectPred: a fully connected network (standard multilayer perceptron) with supervisor heads (one MLP for each target variable) b) Supervised Variational Autoencoder: A variational autoencoder (MMD-loss) with supervisor heads (one MLP for each target variable) c) MultiTripletNetwork: A network structured in triplets to enable contrastive learning (using triplet loss) and additional supervisor heads (one MLP for each target variable)
 3. `target_variables` : A comma separated list of target variables (specify the column headers from the clin.csv).
 - One MLP per each target variable will be created.
 - The target variables may contain NA values
 4. `config_name` : which hyperparameter search space configuration to use.
 5. `n_iter` : How many hyperparameter search steps to implement.
- This example runs 1 hyperparameter search step using DirectPred architecture and a hyperparameter configuration space defined for "DirectPred" with a supervisor head for "CLAUDIN_SUBTYPE" variable:

```

tuner = flexynesis.HyperparameterTuning(dataset = train_dataset, model_class = flexynesis.DirectPred, target_variables =
["CLAUDIN_SUBTYPE"], config_name = "DirectPred", n_iter=1)

```

- We use `perform_tuning` function to run the hyperparameter optimisation procedure. At the end of the parameter optimisation, best model will be selected and returned.

```
model, best_params = tuner.perform_tuning()
```

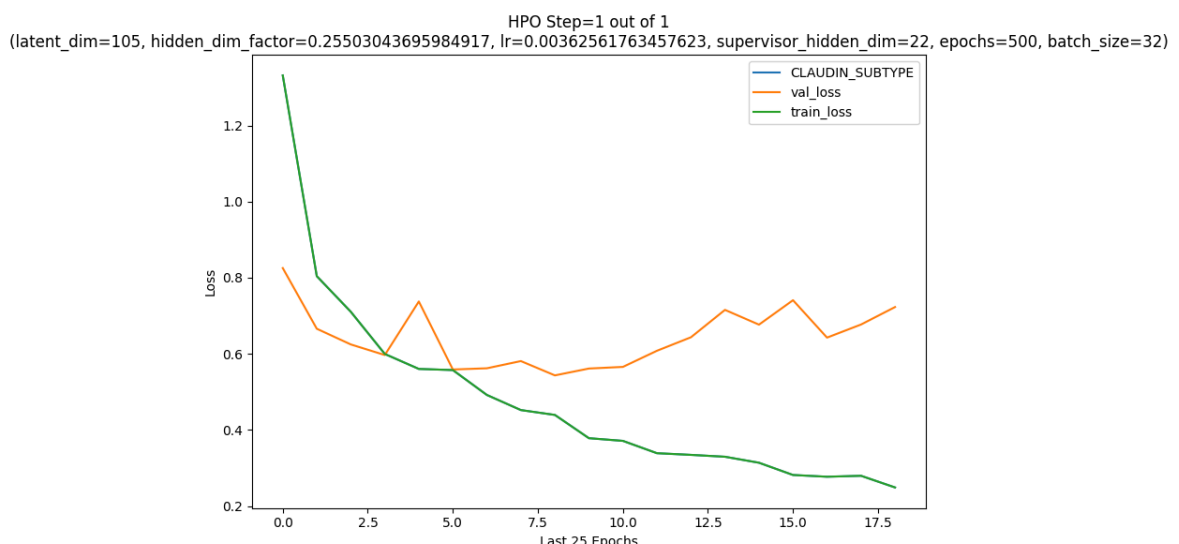
Early Stopping

Training a model longer than needed causes the model to overfit, yield worse validation performance, and also it takes a longer time to train the models, considering if we have to run a long hyperparameter optimisation routine, not just for 1 step, but say more than 100 steps.

It is possible to set `early stopping` criteria in flexynesis, which is basically a simple callback that is handled by `Pytorch Lightning`. This is regulated using the `early_stop_patience`. When set to e.g. 10, the training will stop if the validation loss has not been improved in the last 10 epochs.

One can also visualize the training setting `plot_losses` to `True`. This will print the loss values training/validation splits and also the individual loss values for each target variable. In this case, the total loss value for the training equals the loss value of the single variable we chose.

```
In [13]: tuner = flexynesis.HyperparameterTuning(dataset = train_dataset,
                                                model_class = flexynesis.DirectPr
                                                target_variables = ["CLAUDIN_SUBT
                                                config_name = "DirectPred",
                                                n_iter=1, plot_losses=True, earl
model, best_params = tuner.perform_tuning()
```



Validation: |
?it/s]

| 0/? [00:00<?,

Validate metric	DataLoader 0
CLAUDIN_SUBTYPE	0.7229953408241272
val_loss	0.7229953408241272

```
Tuning Progress: 100%|█| 1/1 [00:08<00:00, 8.48s/it, Iteration=1, Best Loss=0.7
```

```
[INFO] current best val loss: 0.7229953408241272; best params: {'latent_dim': np.int64(105), 'hidden_dim_factor': 0.25503043695984917, 'lr': 0.00362561763457623, 'supervisor_hidden_dim': np.int64(22), 'epochs': 500, 'batch_size': np.int64(32)} since 0 hpo iterations
```

- One can also provide own parameter optimisation spaces via a `yaml` file as input:

```
tuner = flexynesis.HyperparameterTuning(dataset = train_dataset, model_class = flexynesis.DirectPred, target_variables = ["CLAUDIN_SUBTYPE"], config_name = "DirectPred", n_iter=1, plot_losses=True, config_path='./conf.yaml') model, best_params = tuner.perform_tuning()
```

- We can also provide multiple target variables as input. This will create multiple MLP heads (one per variable) and the network will be trained to learn to predict both variables.

```
tuner = flexynesis.HyperparameterTuning(dataset = train_dataset, model_class = flexynesis.DirectPred, target_variables = ["CLAUDIN_SUBTYPE", "CHEMOTHERAPY"], config_name = "DirectPred", n_iter=1, plot_losses=True, early_stop_patience=10) model, best_params = tuner.perform_tuning()
```

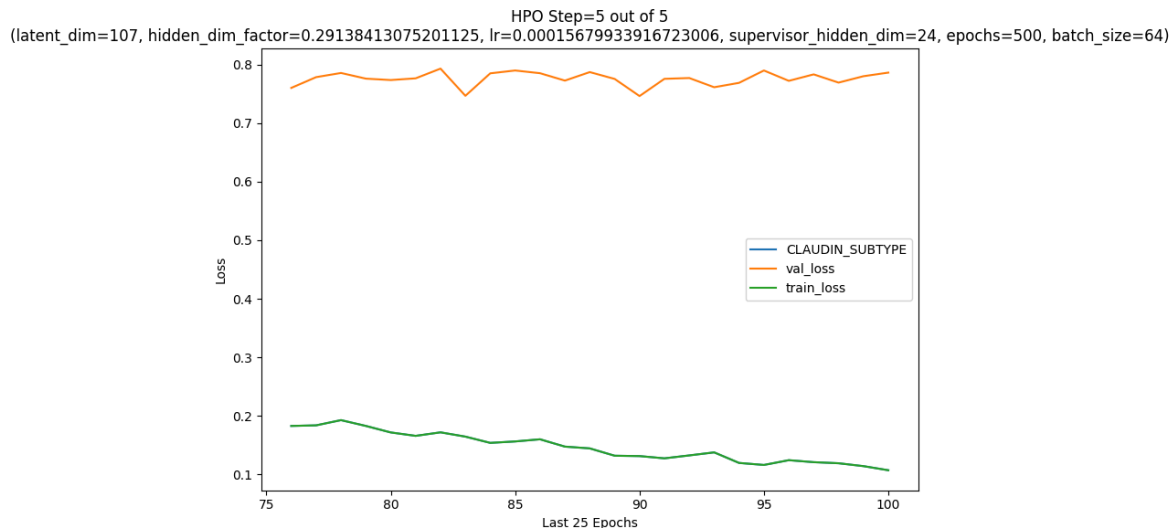
- We can mix numerical and categorical variables. The relevant network structure and evaluation procedures will be applied depending on the type of variable

```
tuner = flexynesis.HyperparameterTuning(dataset = train_dataset, model_class = flexynesis.DirectPred, target_variables = ["CLAUDIN_SUBTYPE", "CHEMOTHERAPY", "LYMPH_NODES_EXAMINED_POSITIVE"], config_name = "DirectPred", n_iter=1, plot_losses=True, early_stop_patience=10) model, best_params = tuner.perform_tuning()
```

Longer Training

In reality, hyperparameter optimisation should run for multiple steps so that the parameter search space is large enough to find a good set. However, for demonstration purposes, we only run it for 5 steps here.

```
In [14]: tuner = flexynesis.HyperparameterTuning(dataset = train_dataset,
                                                model_class = flexynesis.DirectPred,
                                                target_variables = ["CLAUDIN_SUBTYPE", "CHEMOTHERAPY", "LYMPH_NODES_EXAMINED_POSITIVE"],
                                                config_name = "DirectPred",
                                                n_iter=HPO_ITER, plot_losses=True,
                                                early_stop_patience=10)
model, best_params = tuner.perform_tuning()
```



Validation: | 0/? [00:00<?,
?it/s]

Validate metric	DataLoader 0
CLAUDIN_SUBTYPE val_loss	0.7864649295806885 0.7864649295806885

Tuning Progress: 100%|█| 5/5 [01:11<00:00, 14.34s/it, Iteration=5, Best Loss=0.6
[INF0] current best val loss: 0.6479609608650208; best params: {'latent_dim': np.int64(82), 'hidden_dim_factor': 0.21393512381599933, 'lr': 0.0016409286730647932, 'supervisor_hidden_dim': np.int64(12), 'epochs': 500, 'batch_size': np.int64(128)} since 1 hpo iterations

In [15]: model

```

Out[15]: DirectPred(
  (log_vars): ParameterDict( (CLAUDIN_SUBTYPE): Parameter containing:
    [torch.FloatTensor of size 1])
  (encoders): ModuleList(
    (0): MLP(
      (layer_1): Linear(in_features=408, out_features=87, bias=True)
      (layer_out): Linear(in_features=87, out_features=82, bias=True)
      (relu): ReLU()
      (dropout): Dropout(p=0.1, inplace=False)
      (batchnorm): BatchNorm1d(87, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (1): MLP(
      (layer_1): Linear(in_features=450, out_features=96, bias=True)
      (layer_out): Linear(in_features=96, out_features=82, bias=True)
      (relu): ReLU()
      (dropout): Dropout(p=0.1, inplace=False)
      (batchnorm): BatchNorm1d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (MLPs): ModuleDict(
    (CLAUDIN_SUBTYPE): MLP(
      (layer_1): Linear(in_features=164, out_features=12, bias=True)
      (layer_out): Linear(in_features=12, out_features=7, bias=True)
      (relu): ReLU()
      (dropout): Dropout(p=0.1, inplace=False)
      (batchnorm): BatchNorm1d(12, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
)

```

```
In [16]: best_params
```

```

Out[16]: {'latent_dim': np.int64(82),
  'hidden_dim_factor': 0.21393512381599933,
  'lr': 0.0016409286730647932,
  'supervisor_hidden_dim': np.int64(12),
  'epochs': 36,
  'batch_size': np.int64(128)}

```

Prediction and Model Evaluation

We can use the best model (chosen based on the hyperparameter optimisation procedure) to make predictions on the test dataset

```
In [17]: y_pred_dict = model.predict(test_dataset)
```

```
In [18]: y_pred_dict
```



```
Out[18]: {'CLAUDIN_SUBTYPE': array([[1.79400239e-02, 1.52827008e-02, 3.71622711e-01, ...,
1.55110415e-02, 5.10554671e-01, 6.23426996e-02],
[8.93059827e-04, 1.35272148e-03, 9.68172967e-01, ...,
1.85254379e-03, 2.46458948e-02, 2.34083226e-03],
[3.70074064e-01, 8.98678694e-03, 1.94318162e-03, ...,
9.65447631e-03, 2.28377879e-02, 5.67860067e-01],
...,
[7.68337259e-03, 8.24026577e-03, 9.21701491e-01, ...,
5.62545331e-03, 1.02567263e-02, 4.20303643e-03],
[8.38442445e-01, 6.84924945e-02, 3.28584644e-03, ...,
1.12861721e-02, 1.25600351e-02, 4.97416817e-02],
[9.27552059e-02, 6.03938460e-01, 3.70480567e-02, ...,
3.82255986e-02, 9.90455523e-02, 6.14727251e-02]], dtype=float32)}}
```

- The predictions are class labels for both variables. Now, we can run `evaluate_wrapper` to evaluate all predictions. The wrapper goes through each variable and figures out which type of evaluation to apply to the corresponding variable (whether to report metrics relevant to regression tasks or classification tasks)

```
In [19]: metrics_df = flexynesis.evaluate_wrapper(method = 'DirectPred', y_pred_di
metrics_df
```

```
Out[19]:
```

	method	var	variable_type	metric	value
0	DirectPred	CLAUDIN_SUBTYPE	categorical	balanced_acc	0.602674
1	DirectPred	CLAUDIN_SUBTYPE	categorical	f1_score	0.751854
2	DirectPred	CLAUDIN_SUBTYPE	categorical	kappa	0.683135
3	DirectPred	CLAUDIN_SUBTYPE	categorical	average_auroc	0.940547
4	DirectPred	CLAUDIN_SUBTYPE	categorical	average_aupr	0.824016

Extracting the sample embeddings

All models trained within `flexynesis` comes with a `transform` method, which extracts the sample embeddings that are generated by the encoding networks (whether it is an MLP or a variational autoencoder). The embeddings reflect a merged representation of multiple omic layers.

```
In [20]: ds = test_dataset
E = model.transform(ds)
```

```
In [21]: E.head()
```

Out [21]:

	E0	E1	E2	E3	E4	E5	E6
MB-0171	-0.607976	0.963103	-0.769841	-0.435776	0.277644	-0.156224	0.389545
MB-6273	-1.042106	0.527456	-0.183591	-0.799919	0.301279	0.230237	0.428250
MB-5173	-0.254791	1.718872	0.373633	0.514037	0.540497	0.701252	0.342489
MB-2814	-0.007760	0.078139	-0.092678	0.627865	-0.812359	-0.195366	-0.522133
MB-2517	-0.438163	-0.102420	0.254163	-0.005106	0.493878	-0.057032	0.249776

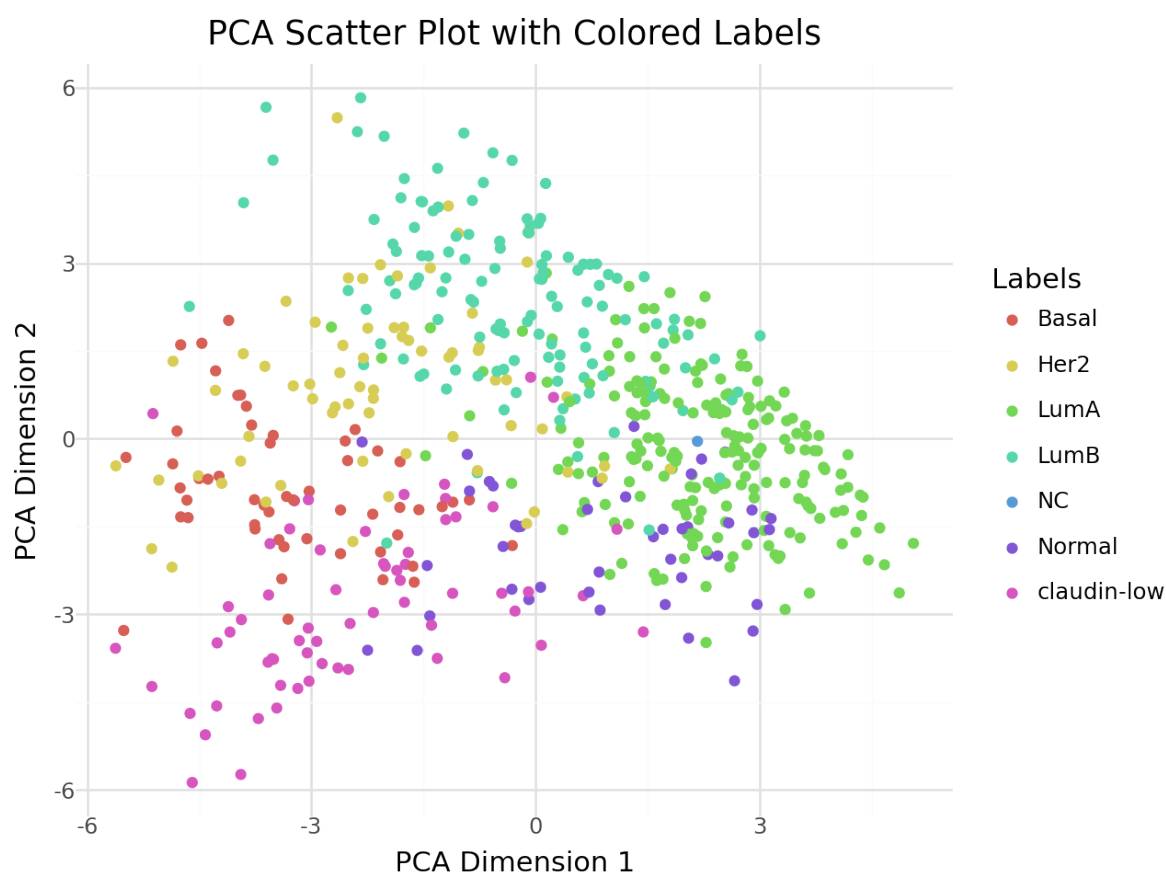
5 rows x 164 columns

Visualizing the sample embeddings

Let's visualize the embeddings in a reduced space and color by the target variables.

```
In [22]: f = 'CLAUDIN_SUBTYPE'
labels = [ds.label_mappings[f][x] for x in ds.ann[f].numpy()] #map the sa
```

```
In [23]: flexynesis.plot_dim_reduced(E, labels, color_type = 'categorical', method
```

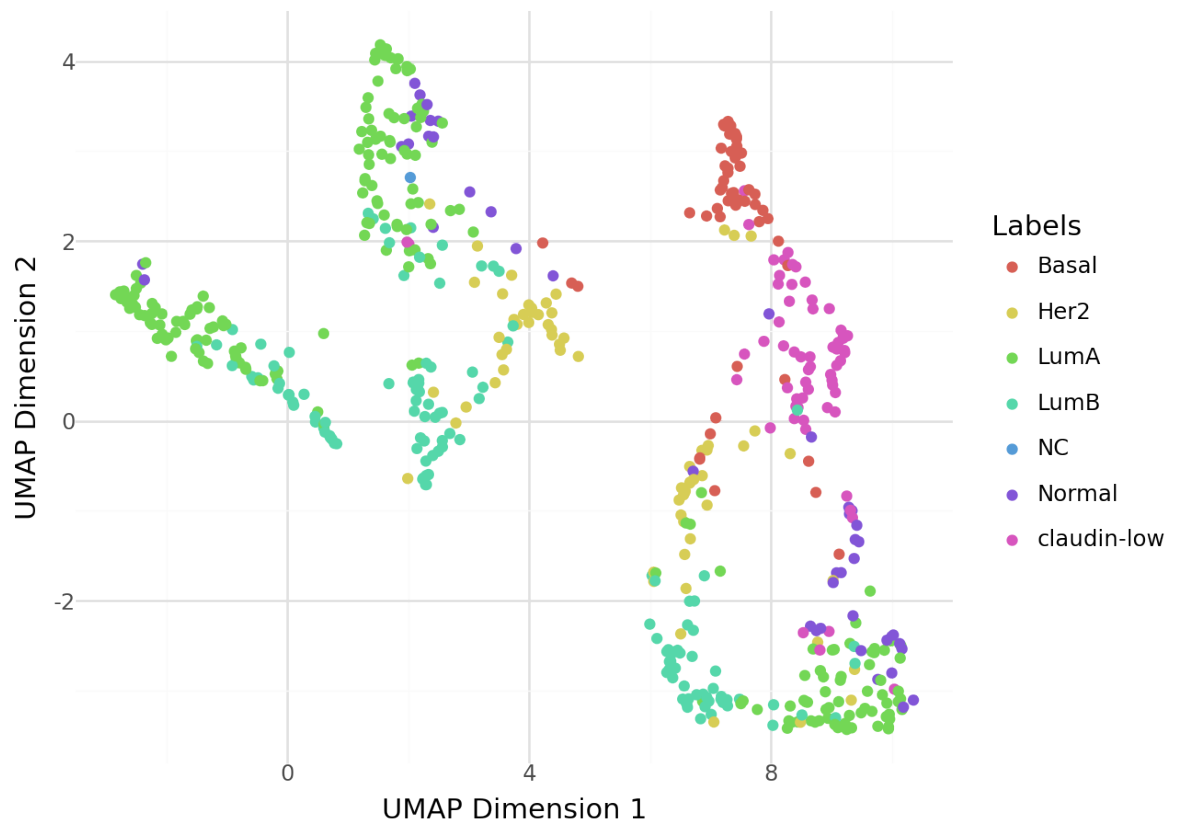


We can also use UMAP visualisation

```
In [24]: flexynesis.plot_dim_reduced(E, labels, color_type = 'categorical', method
```

```
/usr/local/lib/python3.11/site-packages/sklearn/utils/deprecation.py:151:  
FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.  
6 and will be removed in 1.8.
```

UMAP Scatter Plot with Colored Labels



In []: